

integration_architecture

HelixCode [?] LLMsVerifier Integration Architecture

Version: 1.0.0 **Date:** 2026-05-01 **Author:** Integration Architecture Team **Status:** Design Complete — Ready for Implementation

1. Executive Summary

This document specifies the complete integration architecture for making **LLMsVerifier** the single source of truth for model provisioning in **HelixCode**. The design replicates the proven patterns from **HelixAgent**'s `internal/verifier/integration` [33](#), adapted to HelixCode's existing `internal/llm/provider` architecture [15](#).

Core Principle: LLMsVerifier runs as an external service (or embedded goroutine in single-process mode). HelixCode communicates with it via a **REST API client** (not Go module import) to avoid the `digital.vasic.llmprovider` sibling-module dependency hell [16](#). All model listings, provider scores, verification statuses, and pricing flow through this client.

2. Configuration Schema

2.1 Go Struct Additions to `internal/config/config.go`

Modification Point: `helix_code/internal/config/config.go`, after line 253 (after `Cognee *CogneeConfig`)

```
// Config – existing struct, ADD this field:
type Config struct {
    Version      string           `mapstructure:"version"`
    UpdatedBy    string           `mapstructure:"updated_by"`
    Application  ApplicationConfig `mapstructure:"application"`
    Server       ServerConfig      `mapstructure:"server"`
    Database     database.Config    `mapstructure:"database"`
    Redis        RedisConfig        `mapstructure:"redis"`
    Auth         AuthConfig         `mapstructure:"auth"`
    Workers      WorkersConfig      `mapstructure:"workers"`
    Tasks        TasksConfig        `mapstructure:"tasks"`
    LLM          LLMConfig          `mapstructure:"llm"`
    Providers    ProvidersConfig    `mapstructure:"providers"`
    Logging      LoggingConfig      `mapstructure:"logging"`
    Cognee       *CogneeConfig      `mapstructure:"cognee"`
}
```

```

    Verifier      *VerifierConfig
        `mapstructure:"verifier"`    // ← NEW
    HelixAgent    *HelixAgentConfig
        `mapstructure:"helix_agent"` // ← NEW
}

```

New struct definitions (append to `internal/config/config.go` after existing structs):

```

// VerifierConfig controls LLMsVerifier integration.
type VerifierConfig struct {
    Enabled          bool
        `mapstructure:"enabled"`
    Mode             string
        `mapstructure:"mode"`           // "remote" | "embedded"
    Endpoint         string
        `mapstructure:"endpoint"`       // REST API URL
    APIKey           string
        `mapstructure:"api_key"`
    Timeout          time.Duration
        `mapstructure:"timeout"`
    CacheTTL         time.Duration
        `mapstructure:"cache_ttl"`
    PollingInterval  time.Duration
        `mapstructure:"polling_interval"`
    Scoring          VerifierScoringConfig
        `mapstructure:"scoring"`
    Health           VerifierHealthConfig
        `mapstructure:"health"`
    Events           VerifierEventsConfig
        `mapstructure:"events"`
    Providers        map[string]VerifierProviderConfig
        `mapstructure:"providers"`
}

// VerifierScoringConfig mirrors HelixAgent's scoring weights
// [^33^].
type VerifierScoringConfig struct {
    Weights          ScoringWeights `mapstructure:"weights"`
    ModelsDevEnabled bool
        `mapstructure:"models_dev_enabled"`
    ModelsDevEndpoint string
        `mapstructure:"models_dev_endpoint"`
    MinAcceptableScore float64
        `mapstructure:"min_acceptable_score"`
}

type ScoringWeights struct {
    CodeCapability float64
        `mapstructure:"code_capability"` // default: 0.40
}

```

```

    Responsiveness    float64
        `mapstructure:"responsiveness"`    // default: 0.20
    Reliability        float64
        `mapstructure:"reliability"`        // default: 0.20
    FeatureRichness    float64
        `mapstructure:"feature_richness"`    // default: 0.15
    ValueProposition    float64
        `mapstructure:"value_proposition"`    // default: 0.05
}

// VerifierHealthConfig mirrors HelixAgent's circuit breaker
// [^33^].
type VerifierHealthConfig struct {
    CheckInterval    time.Duration
        `mapstructure:"check_interval"`
    Timeout          time.Duration
        `mapstructure:"timeout"`
    FailureThreshold int
        `mapstructure:"failure_threshold"`
    RecoveryThreshold int
        `mapstructure:"recovery_threshold"`
    CircuitBreaker    CircuitBreakerConfig
        `mapstructure:"circuit_breaker"`
}

type CircuitBreakerConfig struct {
    Enabled          bool        `mapstructure:"enabled"`
    HalfOpenTimeout time.Duration
        `mapstructure:"half_open_timeout"`
}

// VerifierEventsConfig controls event publishing.
type VerifierEventsConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    WebSocket  bool    `mapstructure:"websocket"`
    WebSocketPath string `mapstructure:"websocket_path"`
}

// VerifierProviderConfig – per-provider override.
type VerifierProviderConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    APIKey     string `mapstructure:"api_key"`
    BaseURL    string `mapstructure:"base_url"`
    Models     []string `mapstructure:"models"`
    Priority    int     `mapstructure:"priority"`
}

// HelixAgentConfig controls HelixAgent submodule integration.

```

```

type HelixAgentConfig struct {
    Enabled      bool           `mapstructure:"enabled"`
    Path         string          `mapstructure:"path"`           // submodule path
    AutoStart    bool           `mapstructure:"auto_start"`
    VerifierSync HelixAgentVerifierSync `mapstructure:"verifier_sync"`
}

type HelixAgentVerifierSync struct {
    Enabled      bool           `mapstructure:"enabled"`
    ShareScores  bool           `mapstructure:"share_scores"`
    ShareProviders bool          `mapstructure:"share_providers"`
    SyncInterval time.Duration `mapstructure:"sync_interval"`
}

```

2.2 Default Values Function

Modification Point: `helix_code/internal/config/config.go`, inside `setDefault()` after existing defaults.

```

func setDefaults() {
    // ... existing defaults ...

    // LLMsVerifier defaults
    v.SetDefault("verifier.enabled", false)
    v.SetDefault("verifier.mode", "remote")
    v.SetDefault("verifier.endpoint", "http://localhost:8081")
    v.SetDefault("verifier.timeout", "30s")
    v.SetDefault("verifier.cache_ttl", "5m")
    v.SetDefault("verifier.polling_interval", "60s")

    // Scoring defaults (match LLMsVerifier weights [^18^])
    v.SetDefault("verifier.scoring.weights.code_capability",
        0.40)
    v.SetDefault("verifier.scoring.weights.responsiveness", 0.20)
    v.SetDefault("verifier.scoring.weights.reliability", 0.20)
    v.SetDefault("verifier.scoring.weights.feature_richness",
        0.15)
    v.SetDefault("verifier.scoring.weights.value_proposition",
        0.05)
    v.SetDefault("verifier.scoring.min_acceptable_score", 6.0)

    // Health defaults (match HelixAgent [^33^])
    v.SetDefault("verifier.health.check_interval", "30s")
    v.SetDefault("verifier.health.timeout", "10s")
    v.SetDefault("verifier.health.failure_threshold", 5)
    v.SetDefault("verifier.health.recovery_threshold", 3)
}

```

```

v.SetDefault("verifier.health.circuit_breaker.enabled", true)
v.SetDefault("verifier.health.circuit_breaker.half_open_timeout",
    "60s")

// Event defaults
v.SetDefault("verifier.events.enabled", true)
v.SetDefault("verifier.events.websocket", false)
v.SetDefault("verifier.events.websocket_path", "/ws/verifier/
    events")

// HelixAgent defaults
v.SetDefault("helix_agent.enabled", false)
v.SetDefault("helix_agent.path", "./HelixAgent")
v.SetDefault("helix_agent.auto_start", false)
v.SetDefault("helix_agent.verifier_sync.enabled", true)
v.SetDefault("helix_agent.verifier_sync.share_scores", true)
v.SetDefault("helix_agent.verifier_sync.sync_interval", "5m")
}

```

2.3 Environment Variable Bindings

Modification Point: `helix_code/internal/config/config.go`, after existing explicit env var bindings.

```

// Explicitly bind critical env vars (HelixAgent pattern [^33^])
_ = v.BindEnv("verifier.api_key", "HELIX_VERIFIER_API_KEY")
_ = v.BindEnv("verifier.endpoint", "HELIX_VERIFIER_ENDPOINT")
_ = v.BindEnv("verifier.scoring.models_dev_endpoint",
    "HELIX_MODELS_DEV_ENDPOINT")

// Per-provider API keys (HelixAgent .env.example pattern [^33^])
_ = v.BindEnv("verifier.providers.openai.api_key",
    "OPENAI_API_KEY")
_ = v.BindEnv("verifier.providers.anthropic.api_key",
    "ANTHROPIC_API_KEY")
_ = v.BindEnv("verifier.providers.gemini.api_key",
    "GEMINI_API_KEY")
_ = v.BindEnv("verifier.providers.deepseek.api_key",
    "DEEPSEEK_API_KEY")
_ = v.BindEnv("verifier.providers.groq.api_key", "GROQ_API_KEY")
_ = v.BindEnv("verifier.providers.mistral.api_key",
    "MISTRAL_API_KEY")
_ = v.BindEnv("verifier.providers.xai.api_key", "XAI_API_KEY")
_ = v.BindEnv("verifier.providers.cerebras.api_key",
    "CEREBRAS_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.api_key",
    "CLOUDFLARE_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.account_id",
    "CLOUDFLARE_ACCOUNT_ID")

```

```

_ = v.BindEnv("verifier.providers.siliconflow.api_key",
    "SILICONFLOW_API_KEY")
_ = v.BindEnv("verifier.providers.replicate.api_key",
    "REPLICATE_API_TOKEN")
_ = v.BindEnv("verifier.providers.together.api_key",
    "TOGETHER_API_KEY")
_ = v.BindEnv("verifier.providers.openrouter.api_key",
    "OPENROUTER_API_KEY")

```

2.4 Configuration Validation

Modification Point: helix_code/internal/config/config.go, inside validateConfig().

```

func (c *Config) validateConfig() error {
    // ... existing validation ...

    if c.Verifier != nil && c.Verifier.Enabled {
        if c.Verifier.Mode != "remote" && c.Verifier.Mode !=
            "embedded" {
            return fmt.Errorf("verifier.mode must be 'remote' or
                'embedded', got: %s", c.Verifier.Mode)
        }
        if c.Verifier.Endpoint == "" && c.Verifier.Mode ==
            "remote" {
            return
                fmt.Errorf("verifier.endpoint is required when mode is
                    'remote'")
        }
        if c.Verifier.PollingInterval < 10*time.Second {
            return fmt.Errorf("verifier.polling_interval must be
                >= 10s")
        }
        totalWeight := c.Verifier.Scoring.Weights.CodeCapability
            +
                c.Verifier.Scoring.Weights.Responsiveness +
                c.Verifier.Scoring.Weights.Reliability +
                c.Verifier.Scoring.Weights.FeatureRichness +
                c.Verifier.Scoring.Weights.ValueProposition
        if math.Abs(totalWeight-1.0) > 0.001 {
            return fmt.Errorf("verifier scoring weights must sum
                to 1.0, got: %.3f", totalWeight)
        }
    }

    return nil
}

```

2.5 YAML Config Example

New File: helix_code/config/verifier.yaml

```
verifier:
  enabled: true
  mode: remote
  endpoint: "http://localhost:8081"
  api_key: "${HELIX_VERIFIER_API_KEY}"
  timeout: 30s
  cache_ttl: 5m
  polling_interval: 60s

scoring:
  weights:
    code_capability: 0.40
    responsiveness: 0.20
    reliability: 0.20
    feature_richness: 0.15
    value_proposition: 0.05
  min_acceptable_score: 6.0
  models_dev_enabled: true
  models_dev_endpoint: "https://api.models.dev"

health:
  check_interval: 30s
  timeout: 10s
  failure_threshold: 5
  recovery_threshold: 3
  circuit_breaker:
    enabled: true
    half_open_timeout: 60s

events:
  enabled: true
  websocket: false
  websocket_path: "/ws/verifier/events"

providers:
  openai:
    enabled: true
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
  anthropic:
    enabled: true
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
  groq:
    enabled: true
```

```

    api_key: "${GROQ_API_KEY}"
deepseek:
    enabled: true
    api_key: "${DEEPSEEK_API_KEY}"
xai:
    enabled: false
    api_key: "${XAI_API_KEY}"
openrouter:
    enabled: true
    api_key: "${OPENROUTER_API_KEY}"

helix_agent:
    enabled: false
    path: "./HelixAgent"
    auto_start: false
    verifier_sync:
        enabled: true
        share_scores: true
        share_providers: true
        sync_interval: 5m

```

3. Module Boundaries

3.1 New Packages and Files

```

helix_code/
├── internal/
│   ├── verifier/
│   │   ├── client.go           (385 lines) # REST API client for LLMs
│   │   ├── cache.go           (180 lines) # In-memory + Redis model
│   │   ├── health.go          (486 lines) # Health monitor + circuit
│   │   ├── polling.go         (220 lines) # Background polling gorou
│   │   ├── events.go          (200 lines) # Event publisher (HelixCo
│   │   ├── adapter.go         (528 lines) # Score adapter (bridges t
│   │   ├── types.go           (150 lines) # Shared verifier types
│   │   └── doc.go              (30 lines)  # Package documentation
│   ├── llm/
│   │   ├── verifier_integration.go (300 lines) # Wire verifier into exis
│   │   └── ... (existing files modified)
│   └── config/
│       └── config.go            (modified)  # Add VerifierConfig + Hel
├── cmd/
│   └── cli/
│       └── main.go              (modified)  # Replace hardcoded model
├── config/
│   └── verifier.yaml            (257 lines) # Example verifier configu
└── pkg/
    └── sdk/
        └── verifier/

```



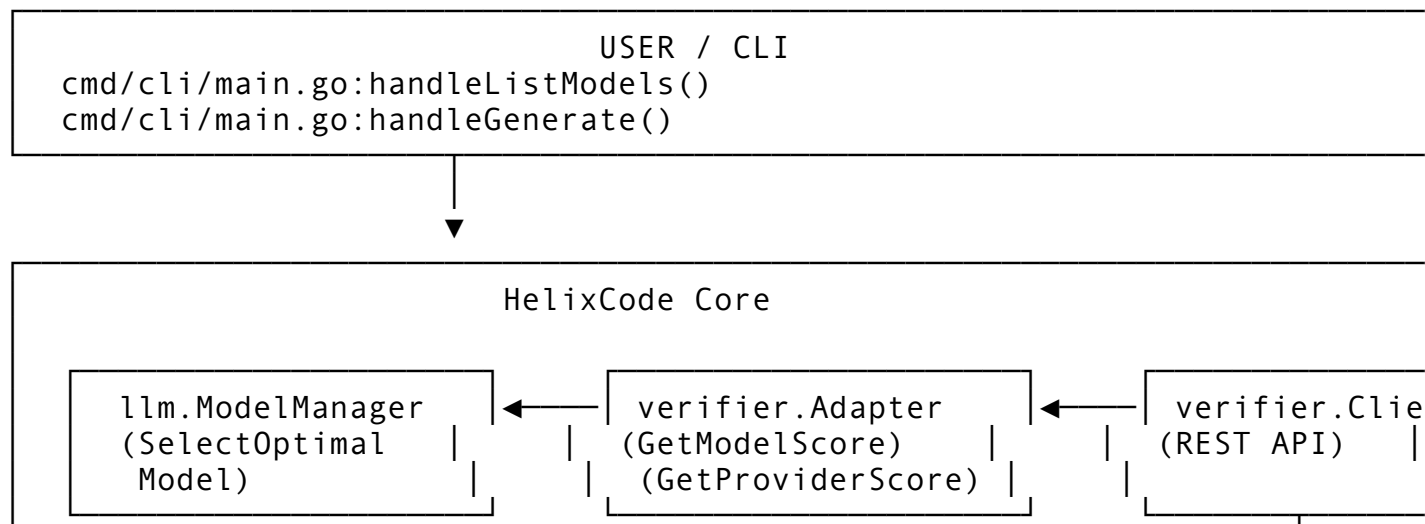
```
└── client.go          (385 lines) # Public SDK (copied from
└── .env.example      (modified)  # Add verifier env vars
```

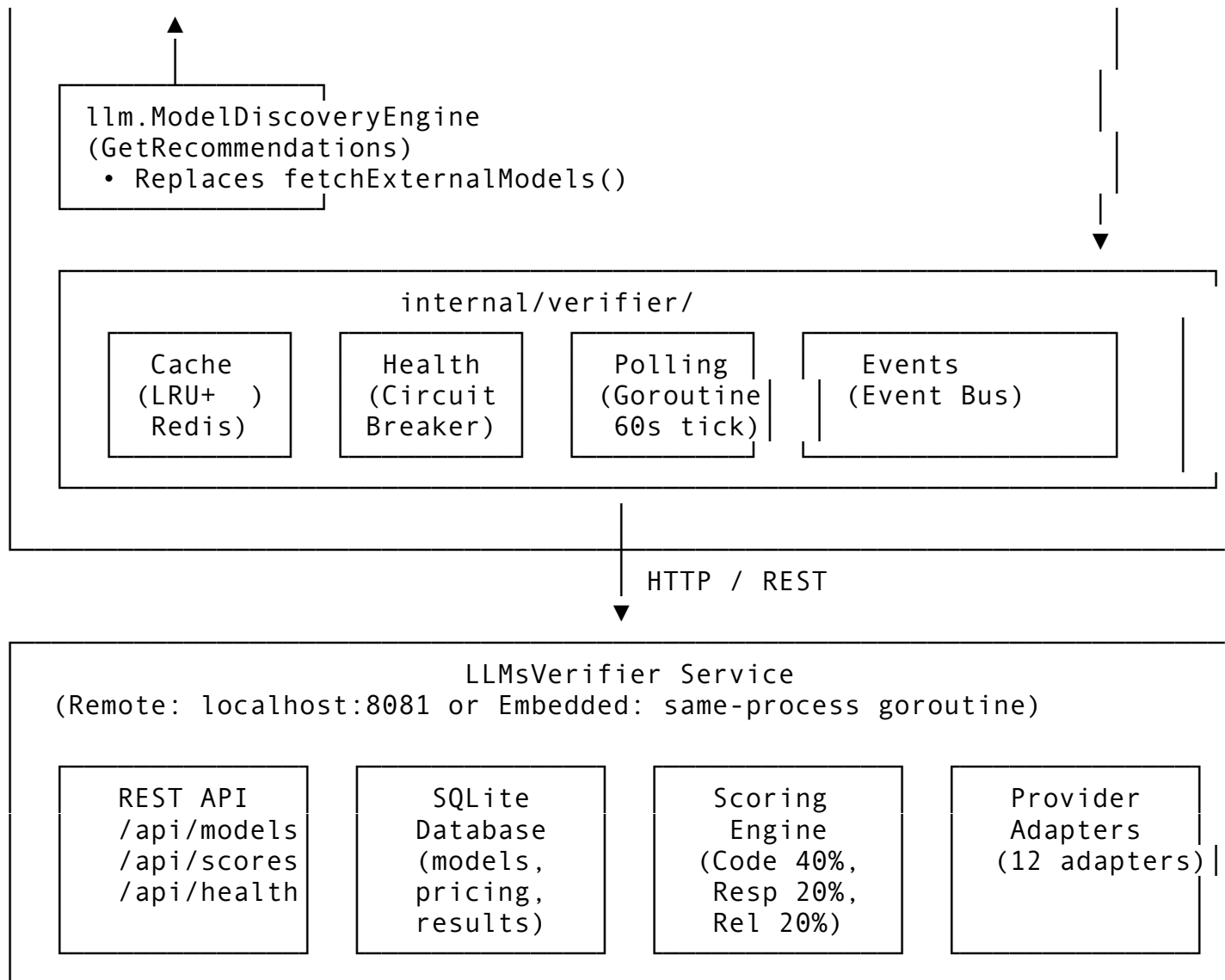
3.2 File Purposes

File	Lines	Responsibility
internal/verifier/ client.go	~385	HTTP client: GetModels(), GetProviders(), GetScores(), VerifyModel()
internal/verifier/ cache.go	~180	Two-tier cache: in-memory LRU (1K entries) + Redis fallback. TTL from config.
internal/verifier/ health.go	~486	Circuit breaker (failure_threshold:5, recovery_threshold:3), health checks
internal/verifier/ polling.go	~220	Poller struct: background goroutine, configurable interval, graceful stop
internal/verifier/ events.go	~200	Publishes to HelixCode's internal/ event/ bus on model/provider changes
internal/verifier/ adapter.go	~528	VerifierScoreAdapter: GetProviderScore(), GetModelScore(), GetVerifiedModels()
internal/verifier/ types.go	~150	VerifiedModel, ProviderScore, VerificationStatus structs
internal/llm/ verifier_integration.go	~300	VerifierModelSource: implements model source interface for model_discovery.go

4. Data Flow

4.1 ASCII Data Flow Diagram





4.2 Caching Strategy

Two-tier cache (mirroring HelixAgent's safe.Store pattern [33](#)):

1. **L1 — In-Memory LRU:** github.com/hashicorp/golang-lru/v2 (already in HelixCode go.mod [3](#))
 - Key: provider::model_id or provider (for provider scores)
 - Value: *CachedModelEntry or *CachedScoreEntry
 - Capacity: 1024 entries
 - TTL: from verifier.cache_ttl (default: 5m)
2. **L2 — Redis** (optional, falls back to memory-only if Redis unavailable)
 - Key: helix:verifier:models:<provider> and helix:verifier:scores:<provider>
 - TTL: same as L1
 - Serialization: JSON

```
// internal/verifier/cache.go
type Cache struct {
    l1    *lru.Cache[string, *CacheEntry]
    l2    *redis.Client    // from HelixCode internal/redis
```

```

    ttl    time.Duration
    mu     sync.RWMutex
}

type CacheEntry struct {
    Models    []*VerifiedModel
    Scores    map[string]float64
    FetchedAt time.Time
    Source    string // "verifier", "fallback", "embedded"
}

func (c *Cache) GetModels(provider string) ([]*VerifiedModel,
    bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()

    // Try L1
    if entry, ok := c.l1.Get(provider); ok {
        if time.Since(entry.FetchedAt) < c.ttl {
            return entry.Models, true
        }
    }

    // Try L2 (Redis)
    if c.l2 != nil {
        data, err := c.l2.Get(ctx,
            "helix:verifier:models:"+provider).Result()
        if err == nil {
            var entry CacheEntry
            if json.Unmarshal([]byte(data), &entry) == nil {
                if time.Since(entry.FetchedAt) < c.ttl {
                    c.l1.Add(provider, &entry) // backfill L1
                    return entry.Models, true
                }
            }
        }
    }
    return nil, false
}

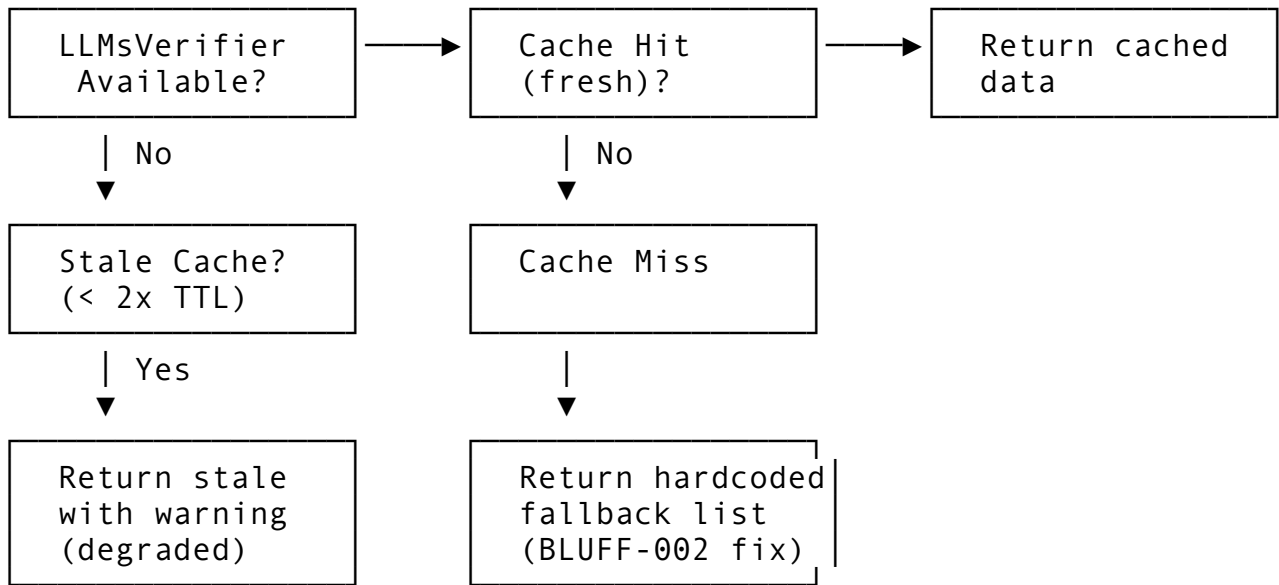
```

4.3 Refresh Intervals

Data Type	L1 Cache TTL	L2 Cache TTL	Polling Interval	Force Refresh Trigger
Model list	5m	5m	60s	User models command, --refresh flag
	5m	10m	60s	

Data Type	L1 Cache TTL	L2 Cache TTL	Polling Interval	Force Refresh Trigger
Provider scores				Health check failure, manual refresh
Verification status	2m	5m	60s	On-demand via <code>VerifyModel()</code>
Pricing	30m	60m	300s	Scheduled re-verification
Provider health	30s	60s	30s	Circuit breaker state change

4.4 Fallback Behavior



Fallback model list (stored in `internal/verifier/fallback_models.go` as `CONST-035` compliance [34](#)):

```
var FallbackModels = []*VerifiedModel{
    {ID: "llama-3.2-3b", Name: "Llama 3.2 3B", Provider:
      "ollama", ContextSize: 131072, Source: "fallback"},
    {ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
      ContextSize: 128000, Source: "fallback"},
    {ID: "claude-3-5-sonnet", Name: "Claude 3.5 Sonnet",
      Provider: "anthropic", ContextSize: 200000, Source:
      "fallback"},
    {ID: "mistral-large", Name: "Mistral Large", Provider:
      "mistral", ContextSize: 128000, Source: "fallback"},
    {ID: "gemini-2.5-pro", Name: "Gemini 2.5 Pro", Provider:
      "gemini", ContextSize: 1000000, Source: "fallback"},
}
```

5. API Contract

5.1 Integration Mode Decision: REST API Client (Recommended)

Decision: HelixCode integrates with LLMsVerifier via **HTTP REST API client**, NOT as an imported Go module.

Rationale: - LLMsVerifier depends on `digital.vasic.llmprovider` at `../..//LLMProvider` [16](#) — this relative module path is incompatible with HelixCode's module structure (`dev.helix.code`). - HelixAgent already exposes a REST API server (`api/server.go` with `/api/models`, `/api/providers`, `/api/health` [28](#)). - REST API allows LLMsVerifier to run as a separate service, sidecar container, or embedded goroutine without module coupling. - HelixAgent's own `pkg/sdk/go/verifier/client.go` [33](#) confirms this is the intended integration pattern.

5.2 REST API Endpoints Consumed

Endpoint	Method	Purpose	Response Type
<code>/api/health</code>	GET	Verifier service health	<code>{"status":"healthy","timestamp":"..."}</code>
<code>/api/models</code>	GET	List all verified models	<code>[]ModelInfo</code> 40
<code>/api/models/{id}</code>	GET	Single model details	<code>ModelInfo</code>
<code>/api/models/{id}/verify</code>	POST	Trigger verification	<code>VerificationResult</code>
<code>/api/providers</code>	GET	List provider statuses	<code>[]ProviderInfo</code>
<code>/api/providers/{id}/scores</code>	GET	Provider scores	<code>{"overall":8.5,"code_capability":8.5,...}</code>
<code>/api/scores</code>	GET	All model scores	<code>map[string]float64</code>
<code>/api/pricing</code>	GET	Token pricing	<code>[]PricingEntry</code>
<code>/api/limits</code>	GET	Rate limits	<code>[]LimitEntry</code>

5.3 Client Implementation

File: helix_code/internal/verifier/client.go

```
package verifier

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

// Client mirrors HelixAgent's pkg/sdk/go/verifier/client.go
// [^33^].
type Client struct {
    baseURL      string
    apiKey       string
    httpClient    *http.Client
    timeout       time.Duration
}

func NewClient(baseURL, apiKey string, timeout time.Duration)
    *Client {
    if timeout == 0 {
        timeout = 30 * time.Second
    }
    return &Client{
        baseURL:      baseURL,
        apiKey:       apiKey,
        timeout:      timeout,
        httpClient: &http.Client{Timeout: timeout},
    }
}

func (c *Client) Health(ctx context.Context) (*HealthResponse,
    error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/api/health", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil,
        fmt.Errorf("verifier health check failed: %w", err) }
    defer resp.Body.Close()
    // ... parse JSON ...
}
```

```

func (c *Client) GetModels(ctx context.Context)
    ([]*VerifiedModel, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/api/models", nil)
    if err != nil { return nil, err }
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, fmt.Errorf("failed to fetch
        models from verifier: %w", err) }
    defer resp.Body.Close()

    var models []*VerifiedModel
    if err := json.NewDecoder(resp.Body).Decode(&models); err !=
        nil {
        return nil, fmt.Errorf("failed to decode models: %w",
            err)
    }
    return models, nil
}

func (c *Client) GetProviderScores(ctx context.Context)
    (map[string]float64, error) {
    req, _ := http.NewRequestWithContext(ctx, "GET", c.baseURL+"/
        api/scores", nil)
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var scores map[string]float64
    json.NewDecoder(resp.Body).Decode(&scores)
    return scores, nil
}

func (c *Client) VerifyModel(ctx context.Context, modelID
    string) (*VerificationResult, error) {
    req, _ := http.NewRequestWithContext(ctx, "POST",
        fmt.Sprintf("%s/api/models/%s/verify", c.baseURL,
            modelID), nil)
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }

```

```

defer resp.Body.Close()

var result VerificationResult
json.NewDecoder(resp.Body).Decode(&result)
return &result, nil
}

```

5.4 Data Types

File: helix_code/internal/verifier/types.go

```

package verifier

import "time"

// VerifiedModel – unified model representation from
// LLMsVerifier.
type VerifiedModel struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    DisplayName   string    `json:"display_name"`
    Provider      string    `json:"provider"`
    ProviderType  string    `json:"provider_type"`
    Score         float64   `json:"score"`
    Verified      bool      `json:"verified"`
    VerificationStatus string  `json:"verification_status"` //
    pending, verified, failed
    ContextSize   int       `json:"context_window_tokens"`
    MaxOutputTokens int     `json:"max_output_tokens"`
    SupportsStreaming bool    `json:"supports_streaming"`
    SupportsTools bool    `json:"supports_tool_use"`
    SupportsCode  bool      `json:"supports_code_generation"`
    SupportsVision bool     `json:"supports_vision"`
    Latency       time.Duration `json:"latency_ms"`
    CostPerInputToken float64 `json:"input_token_cost"`
    CostPerOutputToken float64 `json:"output_token_cost"`
    OverallScore   float64 `json:"overall_score"`
    CodeCapabilityScore float64 `json:"code_capability_score"`
    ResponsivenessScore float64 `json:"responsiveness_score"`
    ReliabilityScore float64 `json:"reliability_score"`
    FeatureRichnessScore float64 `json:"feature_richness_score"`
    ValuePropositionScore float64 `json:"value_proposition_score"`
    LastVerified     time.Time `json:"last_verified"`
    Source           string    `json:"source"` // "verifier",
    "cache", "fallback"
}

```



```
// ProviderStatus – health and score of a provider.
type ProviderStatus struct {
    Name      string `json:"name"`
    Type      string `json:"type"`
    Score     float64 `json:"score"`
    Verified  bool   `json:"verified"`
    Healthy   bool   `json:"healthy"`
    ModelCount int    `json:"model_count"`
    Tier      int    `json:"tier"`
    LastChecked time.Time `json:"last_checked"`
}

// VerificationResult – result of on-demand verification.
type VerificationResult struct {
    ModelID      string `json:"model_id"`
    Status       string `json:"status"`
    OverallScore float64 `json:"overall_score"`
    Error        string `json:"error,omitempty"`
    CompletedAt  time.Time `json:"completed_at"`
}

// HealthResponse – verifier service health.
type HealthResponse struct {
    Status      string `json:"status"`
    Timestamp   time.Time `json:"timestamp"`
    Version     string `json:"version"`
}
```

6. Integration Points with Existing Code

6.1 cmd/cli/main.go:handleListModels() — BLUFF-002 Fix

Current Code (lines 101-128) ^[10]:

```
// HARDCODED – REPLACES THIS:
func (c *CLI) handleListModels(ctx context.Context) error {
    models := []struct{...}{
        {"llama-3-8b", "Llama 3 8B", "llama.cpp", 8192,
        "available"},
        {"mistral-7b", "Mistral 7B", "ollama", 4096,
        "available"},
        {"phi-3-mini", "Phi-3 Mini", "openai", 128000,
        "available"},
    }
    // ... prints hardcoded list
}
```

Replacement (lines 101-128 rewritten):

```
func (c *CLI) handleListModels(ctx context.Context) error {
    // Use verifier adapter if enabled, otherwise fall through to
    // model manager
    if c.verifierAdapter != nil && c.verifierAdapter.IsEnabled()
    {
        models, err := c.verifierAdapter.GetVerifiedModels(ctx)
        if err == nil && len(models) > 0 {
            return c.printVerifiedModels(models)
        }
        // Log warning but don't fail – try fallback
        c.logger.Warn("Verifier model fetch failed, using
        fallback: %v", err)
    }

    // Fallback to existing ModelManager (already initialized,
    // may have local models)
    if c.modelManager != nil {
        models := c.modelManager.GetAvailableModels()
        return c.printManagerModels(models)
    }

    // Ultimate fallback: static list (for bootstrapping before
    // any provider is ready)
    return c.printFallbackModels()
}

func (c *CLI) printVerifiedModels(models
    []*verifier.VerifiedModel) error {
    fmt.Println("Available Models (from LLMsVerifier):")
    fmt.Println(strings.Repeat("-", 80))
    fmt.Printf("%-24s %-20s %-10s %-12s %s\n", "ID", "Name",
        "Provider", "Score", "Status")
    for _, m := range models {
        status := "✓ verified"
        if !m.Verified {
            status = "○ pending"
        }
        if m.VerificationStatus == "failed" {
            status = "× failed"
        }
        scoreStr := fmt.Sprintf("SC:%.1f", m.OverallScore)
        fmt.Printf("%-24s %-20s %-10s %-12s %s\n", m.ID, m.Name,
            m.Provider, scoreStr, status)
    }
    return nil
}
```

6.2 internal/llm/model_discovery.go:fetchExternalModels() — Replace Hardcoded List

Current Code [29](#):

```
func (e *ModelDiscoveryEngine) fetchExternalModels()
    []*ModelInfo {
    return []*ModelInfo{
        {ID: "llama-3-8b-instruct", Name: "Llama 3 8B Instruct",
        Format: FormatGGUF, Size: 4.7GB, ContextSize: 8192},
        {ID: "mistral-7b-instruct", Name: "Mistral 7B Instruct",
        Format: FormatGGUF, Size: 4.1GB, ContextSize: 32768},
        {ID: "codellama-7b-instruct", Name: "CodeLlama 7B
        Instruct", Format: FormatGGUF, Size: 3.8GB, ContextSize:
        16384},
    }
}
```

Replacement:

```
func (e *ModelDiscoveryEngine) fetchExternalModels(ctx
    context.Context) []*ModelInfo {
    // If verifier adapter is available and enabled, use it as
    // the single source of truth
    if e.verifierAdapter != nil && e.verifierAdapter.IsEnabled()
    {
        verifiedModels, err :=
        e.verifierAdapter.GetVerifiedModels(ctx)
        if err == nil {
            return e.convertVerifiedToModelInfo(verifiedModels)
        }

        e.logger.Warn("Verifier fetch failed for external models:
        %v", err)
    }

    // Fallback: return empty – local providers (Ollama,
    // llama.cpp) will still be discovered
    return []*ModelInfo{}
}

func (e *ModelDiscoveryEngine)
    convertVerifiedToModelInfo(verified
        []*verifier.VerifiedModel) []*ModelInfo {
    result := make([]*ModelInfo, 0, len(verified))
    for _, v := range verified {
        mi := &ModelInfo{
            ID:          v.ID,
            Name:         v.DisplayName,
        }
    }
}
```

```

        Format:
        FormatUnknown, // Verifier doesn't track GGUF vs HF
        Size:          0, // Not provided by
        verifier
        ContextSize: v.ContextSize,
        MaxTokens:   v.MaxOutputTokens,
        Provider:    v.Provider,
        Verified:    v.Verified,
        Score:       v.OverallScore,
        Capabilities: e.mapCapabilities(v),
    }
    result = append(result, mi)
}
return result
}

```

New Field in ModelDiscoveryEngine (add to `internal/llm/model_discovery.go` struct):

```

type ModelDiscoveryEngine struct {
    // ... existing fields ...
    verifierAdapter *verifier.Adapter // ← NEW
}

```

6.3 `internal/llm/model_manager.go:SelectOptimalModel()` — Use Verifier Scores

Current Scoring Algorithm [33](#):

```

func (m *ModelManager) SelectOptimalModel(criteria
    ModelSelectionCriteria) (*ModelInfo, error) {
    // 1. Capability matching
    // 2. Context size adequacy
    // 3. Task type suitability
    // 4. Hardware compatibility
    // 5. Quality preference
}

```

Augmented Algorithm (insert after step 3, before hardware compatibility):

```

func (m *ModelManager) SelectOptimalModel(criteria
    ModelSelectionCriteria) (*ModelInfo, error) {
    candidates :=
        m.filterByCapabilities(criteria.RequiredCapabilities)
    candidates = m.filterByContextSize(candidates,
        criteria.MaxTokens)
    candidates = m.filterByTaskType(candidates,
        criteria.TaskType)
}

```

```

// NEW: Incorporate LLMsVerifier scores into ranking
if m.verifierAdapter != nil && m.verifierAdapter.IsEnabled()
{
    candidates = m.rankByVerifierScores(candidates, criteria)
}

candidates = m.filterByHardwareCompatibility(candidates)
candidates = m.applyQualityPreference(candidates,
    criteria.QualityPreference)

if len(candidates) == 0 {
    return nil, ErrNoSuitableModel
}
return candidates[0], nil
}

```

```

func (m *ModelManager) rankByVerifierScores(
    candidates []*ModelInfo,
    criteria ModelSelectionCriteria,
) []*ModelInfo {
    scored := make([]struct {
        model *ModelInfo
        score float64
    }, 0, len(candidates))

    for _, c := range candidates {
        baseScore := c.Score // existing heuristic score

        // Fetch verifier score for this specific model
        verifierScore, found :=
            m.verifierAdapter.GetModelScore(c.ID)
        if found {
            // Weighted blend: 60% verifier, 40% local heuristic
            // Verifier score is 0-10; normalize to 0-1
            normalizedVerifier := verifierScore / 10.0
            blendedScore := (normalizedVerifier * 0.6) +
                (baseScore * 0.4)

            // Apply task-specific boost from verifier dimensions
            switch criteria.TaskType {
            case "code_generation", "debugging", "refactoring":
                codeScore, _ :=
                    m.verifierAdapter.GetModelCodeCapabilityScore(c.ID)
                blendedScore += (codeScore / 10.0) *
                    0.15 // +15% code capability bonus
            case "planning", "analysis":
                relScore, _ :=
                    m.verifierAdapter.GetModelReliabilityScore(c.ID)

```

```

        blendedScore += (relScore / 10.0) * 0.10 // +10%
        reliability bonus
    }

    baseScore = blendedScore
}

scored = append(scored, struct {
    model *ModelInfo
    score float64
}{c, baseScore})
}

sort.Slice(scored, func(i, j int) bool {
    return scored[i].score > scored[j].score
}))

result := make([]*ModelInfo, len(scored))
for i, s := range scored {
    result[i] = s.model
}
return result
}

```

6.4 internal/llm/factory.go:NewProvider() — Verifier-Aware Validation

Modification Point: helix_code/internal/llm/factory.go, after provider creation, before return.

```

func NewProvider(config ProviderConfigEntry) (Provider, error) {
    var provider Provider
    var err error

    switch config.Type {
    case ProviderTypeOpenAI:    provider, err =
        NewOpenAIProvider(config)
    case ProviderTypeAnthropic: provider, err =
        NewAnthropicProvider(config)
    // ... existing cases ...
    default:
        return nil, fmt.Errorf("unsupported provider type: %s",
            config.Type)
    }

    if err != nil {
        return nil, err
    }
}

```

```

// NEW: Validate provider against LLMsVerifier registry
if verifierAdapter != nil && verifierAdapter.IsEnabled() {
    status, found :=
        verifierAdapter.GetProviderStatus(string(config.Type))
    if found {
        if !status.Healthy {
            return nil, fmt.Errorf("provider %s is marked
unhealthy by verifier (score: %.1f)",
                config.Type, status.Score)
        }
        if status.Score <
            verifierAdapter.GetMinAcceptableScore() {
            return nil, fmt.Errorf("provider %s score %.1f
below minimum %.1f",
                config.Type, status.Score,
                verifierAdapter.GetMinAcceptableScore())
        }
    }
}

return provider, nil
}

```

6.5 internal/llm/verifier_integration.go — New Bridge File

New File: helix_code/internal/llm/verifier_integration.go

```

package llm

import "dev.helix.code/internal/verifier"

// VerifierModelSource implements the model source interface for
// the discovery engine.
// It delegates to the verifier adapter to fetch real-time model
// data.
type VerifierModelSource struct {
    adapter *verifier.Adapter
}

func NewVerifierModelSource(adapter *verifier.Adapter)
    *VerifierModelSource {
    return &VerifierModelSource{adapter: adapter}
}

func (s *VerifierModelSource) IsAvailable() bool {
    return s.adapter != nil && s.adapter.IsEnabled() &&
        s.adapter.IsReachable()
}

```

```

func (s *VerifierModelSource) FetchModels(ctx context.Context)
    ([]*ModelInfo, error) {
    verified, err := s.adapter.GetVerifiedModels(ctx)
    if err != nil {
        return nil, err
    }
    return s.convert(verified), nil
}

func (s *VerifierModelSource) convert(verified
    []*verifier.VerifiedModel) []*ModelInfo {
    // ... same as
    ModelDiscoveryEngine.convertVerifiedToModelInfo ...
}

```

7. Enable/Disable Mechanism

7.1 Global Enable/Disable

Control Point: internal/config/config.go:VerifierConfig.Enabled

Behavior Matrix:

verifier.enabled	System Behavior
false (default)	Verifier client is nil. All model operations use existing local/heuristic logic. handleListModel() falls through to modelManager.GetAvailableModels(). No polling goroutine started. No REST API calls made.
true	Verifier client initialized. Polling goroutine started. All model lists fetched from verifier first, with local fallback. Scores incorporated into SelectOptimalModel(). Events published on changes.

Initialization Logic (in cmd/server/main.go and cmd/cli/main.go):

```

func initializeVerifier(cfg *config.Config) (*verifier.Adapter,
    error) {
    if cfg.Verifier == nil || !cfg.Verifier.Enabled {
        return nil, nil // disabled – return nil adapter
    }

    client := verifier.NewClient(cfg.Verifier.Endpoint,
        cfg.Verifier.APIKey, cfg.Verifier.Timeout)
    cache := verifier.NewCache(cfg.Verifier.CacheTTL,
        redisClient) // redisClient may be nil
}

```



```

health := verifier.NewHealthMonitor(cfg.Verifier.Health)

adapter := verifier.NewAdapter(client, cache, health,
    cfg.Verifier)

// Start background polling
if cfg.Verifier.PollingInterval > 0 {
    poller := verifier.NewPoller(adapter,
        cfg.Verifier.PollingInterval)
    poller.Start()
    // Store poller for graceful shutdown
}

return adapter, nil
}

```

7.2 Per-Provider Enable/Disable

Control Point: internal/config/
config.go:VerifierProviderConfig.Enabled

Behavior: The verifier adapter filters providers before returning to HelixCode:

```

func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*verifier.VerifiedModel, error) {
    allModels, err := a.client.GetModels(ctx)
    if err != nil { return nil, err }

    filtered := make([]*verifier.VerifiedModel, 0,
        len(allModels))
    for _, m := range allModels {
        // Check per-provider enable flag
        providerCfg, hasOverride :=
            a.config.Providers[m.Provider]
        if hasOverride && !providerCfg.Enabled {
            continue // skip disabled providers
        }

        // Check global provider enable in HelixCode
        ProvidersConfig
        if !a.isProviderEnabledInHelixCode(m.Provider) {
            continue
        }

        filtered = append(filtered, m)
    }
    return filtered, nil
}

```

Provider Enable State Resolution:

verifier.providers.X .enabled	×	helixcode.providers.X .enabled	=	Effective
true true false false (not set) (not set)		true false true false true false		ENABLED DISABLED DISABLED DISABLED ENABLED (in DISABLED (i

7.3 Graceful Degradation When Disabled or Unavailable

```
func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*verifier.VerifiedModel, error) {
    if !a.enabled {
        return nil, ErrVerifierDisabled
    }

    // Try cache first
    if models, ok := a.cache.GetModels("all"); ok {
        return models, nil
    }

    // Try live fetch
    models, err := a.client.GetModels(ctx)
    if err != nil {
        // Circuit breaker may be open
        if a.health.IsCircuitOpen() {
            // Return stale cache if available (even past TTL)
            if stale, ok := a.cache.GetModelsStale("all"); ok {
                return stale, ErrUsingStaleCache
            }
        }
        // Return fallback list
        return verifier.FallbackModels, ErrVerifierUnavailable
    }

    a.cache.SetModels("all", models)
    return models, nil
}
```

8. Real-Time Updates Strategy

8.1 Polling Architecture

Since LLMsVerifier **only supports polling** (no webhooks, no SSE, no push) [28](#), HelixCode implements a background polling goroutine.

```
// internal/verifier/polling.go

type Poller struct {
    adapter      *Adapter
    interval     time.Duration
    ticker       *time.Ticker
    stopCh       chan struct{}
    wg           sync.WaitGroup
    lastModels   map[string]*VerifiedModel
    lastScores   map[string]float64
    mu           sync.RWMutex
}

func NewPoller(adapter *Adapter, interval time.Duration) *Poller
{
    if interval < 10*time.Second {
        interval = 10 * time.Second
    }
    return &Poller{
        adapter:  adapter,
        interval: interval,
        stopCh:   make(chan struct{}),
    }
}

func (p *Poller) Start() {
    p.wg.Add(1)
    go p.loop()
}

func (p *Poller) Stop() {
    close(p.stopCh)
    p.wg.Wait()
}

func (p *Poller) loop() {
    defer p.wg.Done()

    // Immediate first poll
    p.poll()
}
```

```

p.ticker = time.NewTicker(p.interval)
defer p.ticker.Stop()

for {
    select {
    case <-p.ticker.C:
        p.poll()
    case <-p.stopCh:
        return
    }
}

}

func (p *Poller) poll() {
    ctx, cancel := context.WithTimeout(context.Background(),
        30*time.Second)
    defer cancel()

    // 1. Fetch all models
    models, err := p.adapter.client.GetModels(ctx)
    if err != nil {
        p.adapter.health.RecordFailure()
        return
    }

    // 2. Detect changes
    p.mu.Lock()
    changes := p.detectChanges(p.lastModels, models)
    p.lastModels = p.indexModels(models)
    p.mu.Unlock()

    // 3. Update cache
    p.adapter.cache.SetModels("all", models)
    p.adapter.health.RecordSuccess()

    // 4. Publish events for changes
    for _, change := range changes {
        p.adapter.events.Publish(change)
    }

    // 5. Fetch scores (less frequently – every 3rd poll)
    if p.shouldFetchScores() {
        scores, _ := p.adapter.client.GetProviderScores(ctx)
        p.adapter.cache.SetScores(scores)
    }
}

```

```

func (p *Poller) detectChanges(old, new []*VerifiedModel)
    []ChangeEvent {
    changes := []ChangeEvent{}
    newIndex := p.indexModels(new)

    for id, model := range newIndex {
        if oldModel, ok := old[id]; !ok {
            changes = append(changes, ChangeEvent{Type:
                "model.discovered", Model: model})
        } else if oldModel.OverallScore != model.OverallScore {
            changes = append(changes, ChangeEvent{Type:
                "model.score_changed", Model: model, OldScore:
                oldModel.OverallScore})
        } else if oldModel.VerificationStatus !=
            model.VerificationStatus {
            changes = append(changes, ChangeEvent{Type:
                "model.status_changed", Model: model, OldStatus:
                oldModel.VerificationStatus})
        }
    }

    for id, model := range old {
        if _, ok := newIndex[id]; !ok {
            changes = append(changes, ChangeEvent{Type:
                "model.removed", Model: model})
        }
    }

    return changes
}

func (p *Poller) shouldFetchScores() bool {
    // Scores change slower than model lists; poll every 3rd tick
    return time.Now().Unix()%3 == 0
}

```

8.2 Event Publishing to HelixCode Event Bus

File: internal/verifier/events.go

```

package verifier

import "dev.helix.code/internal/event"

const (
    TopicVerifierModelDiscovered =
        "helix.verifier.model.discovered"
    TopicVerifierModelUpdated    = "helix.verifier.model.updated"
    TopicVerifierModelRemoved    = "helix.verifier.model.removed"

```

```

    TopicVerifierProviderHealth =
        "helix.verifier.provider.health"
    TopicVerifierScoreChanged   = "helix.verifier.score.changed"
    TopicVerifierDegraded       = "helix.verifier.degraded"
    TopicVerifierRecovered      = "helix.verifier.recovered"
)

type EventPublisher struct {
    bus      event.Bus
    enabled   bool
    websocket bool
    wsPath    string
}

func (ep *EventPublisher) Publish(change ChangeEvent) error {
    if !ep.enabled {
        return nil
    }

    var topic string
    switch change.Type {
    case "model.discovered":
        topic = TopicVerifierModelDiscovered
    case "model.score_changed":
        topic = TopicVerifierScoreChanged
    case "model.status_changed":
        topic = TopicVerifierModelUpdated
    case "model.removed":
        topic = TopicVerifierModelRemoved
    }

    data, _ := json.Marshal(change)
    return ep.bus.Publish(topic, data)
}

```

8.3 On-Demand Refresh Triggers

Users can force a refresh outside the polling interval:

```

// cmd/cli/main.go — add new flag
var (
    refreshModels = flag.Bool("refresh-models", false, "Force
        refresh model list from verifier")
)

func (c *CLI) handleListModels(ctx context.Context) error {
    if *refreshModels && c.verifierAdapter != nil {
        c.verifierAdapter.ForceRefresh(ctx)
    }
}

```

```
    // ... continue with normal flow ...  
}
```

9. Error Handling & Fallbacks

9.1 Circuit Breaker

File: internal/verifier/health.go

```
type CircuitState int  
  
const (  
    CircuitClosed CircuitState = iota  
    CircuitOpen  
    CircuitHalfOpen  
)  
  
type HealthMonitor struct {  
    state          CircuitState  
    failures       int  
    successes      int  
    lastFailureTime time.Time  
    config         config.VerifierHealthConfig  
    mu             sync.RWMutex  
}  
  
func (h *HealthMonitor) RecordFailure() {  
    h.mu.Lock()  
    defer h.mu.Unlock()  
  
    h.failures++  
    h.lastFailureTime = time.Now()  
  
    if h.config.CircuitBreaker.Enabled && h.failures >=  
        h.config.FailureThreshold {  
        h.state = CircuitOpen  
    }  
}  
  
func (h *HealthMonitor) RecordSuccess() {  
    h.mu.Lock()  
    defer h.mu.Unlock()  
  
    switch h.state {  
    case CircuitHalfOpen:  
        h.successes++  
        if h.successes >= h.config.RecoveryThreshold {
```

```

        h.state = CircuitClosed
        h.failures = 0
        h.successes = 0
    }
    case CircuitClosed:
        h.failures = 0 // reset on success
    }
}

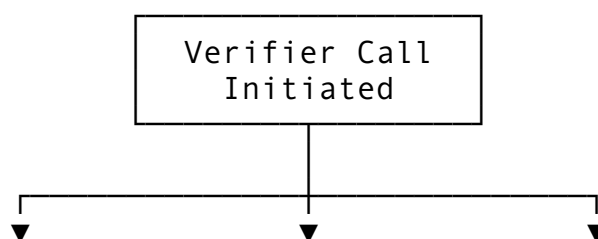
func (h *HealthMonitor) AllowRequest() bool {
    h.mu.RLock()
    defer h.mu.RUnlock()

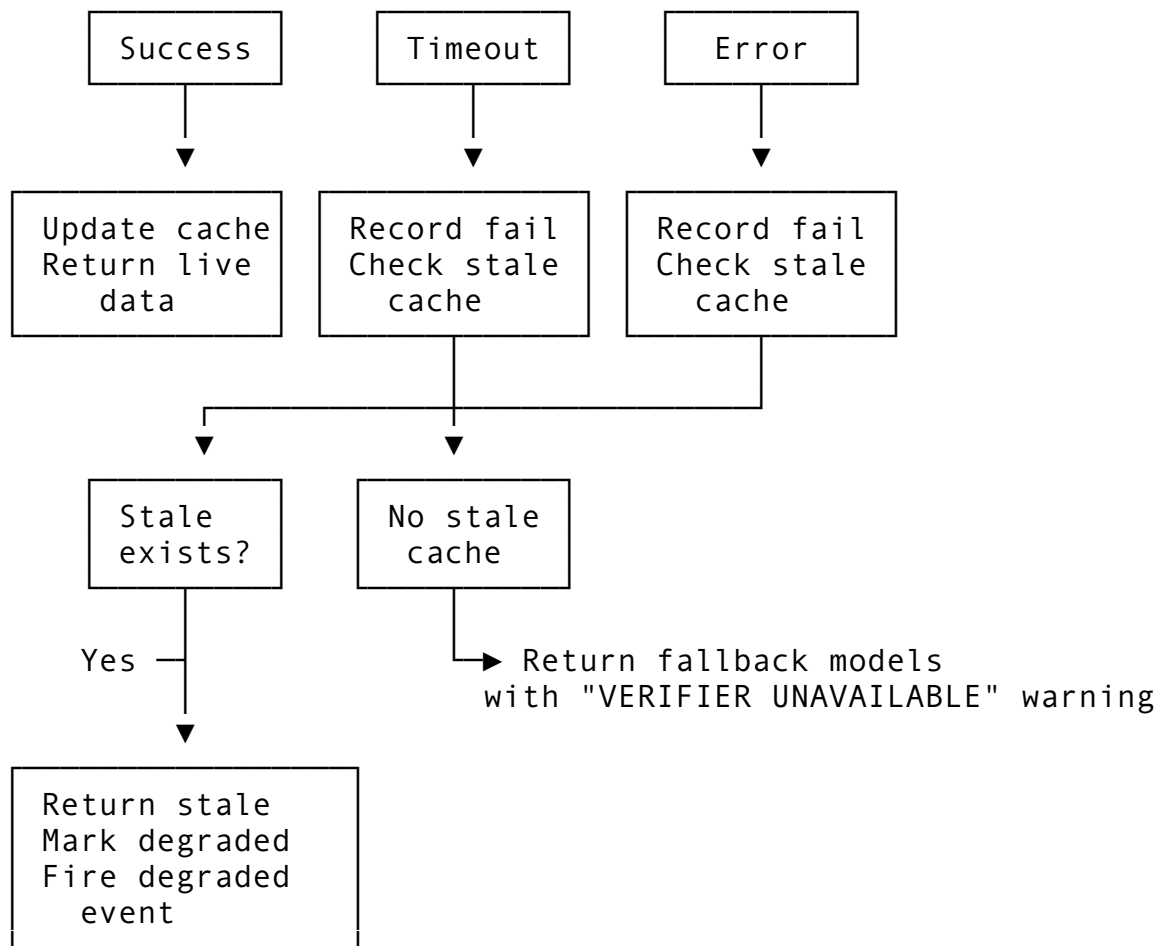
    switch h.state {
    case CircuitClosed:
        return true
    case CircuitOpen:
        if time.Since(h.lastFailureTime) >
            h.config.CircuitBreaker.HalfOpenTimeout {
            h.mu.RUnlock()
            h.mu.Lock()
            h.state = CircuitHalfOpen
            h.successes = 0
            h.mu.Unlock()
            h.mu.RLock()
            return true
        }
        return false
    case CircuitHalfOpen:
        return true
    }
    return false
}

func (h *HealthMonitor) IsCircuitOpen() bool {
    h.mu.RLock()
    defer h.mu.RUnlock()
    return h.state == CircuitOpen
}

```

9.2 Degraded Mode Decision Tree





9.3 Fallback Chain (CONST-020 Compliance) [34](#)

```

// internal/verifier/adapters.go
func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*VerifiedModel, error) {
    if !a.enabled {
        return nil, ErrVerifierDisabled
    }

    if !a.health.AllowRequest() {
        // Circuit open – use cache or fallback
        return a.getFallbackModels()
    }

    // 1. Try live fetch
    models, err := a.client.GetModels(ctx)
    if err == nil {
        a.health.RecordSuccess()
        a.cache.SetModels("all", models)
        return models, nil
    }

    a.health.RecordFailure()

```

```

// 2. Try stale cache (up to 2x TTL)
if stale, ok := a.cache.GetModelsStale("all"); ok {
    return stale, ErrUsingStaleCache
}

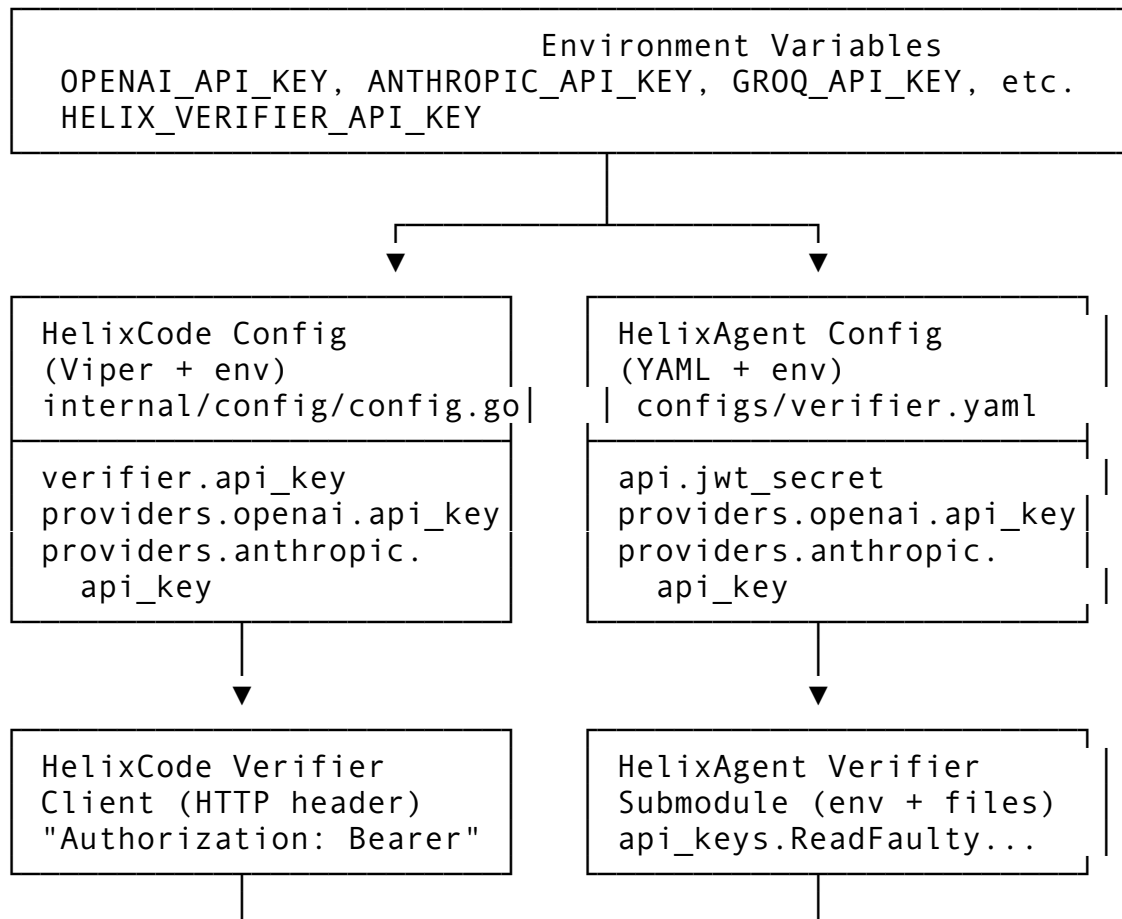
// 3. Return hardcoded fallback
return a.getFallbackModels()
}

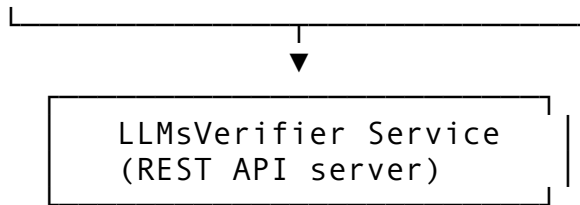
func (a *Adapter) getFallbackModels() ([]*VerifiedModel, error) {
    result := make([]*VerifiedModel, len(FallbackModels))
    for i, m := range FallbackModels {
        copy := *m
        copy.Source = "fallback"
        result[i] = &copy
    }
    return result, ErrUsingFallback
}

```

10. API Key Provisioning

10.1 Key Flow Architecture





10.2 API Key Redaction

Rule: API keys are NEVER serialized to JSON, NEVER logged at INFO or above, NEVER returned in API responses.

```
// internal/config/config.go
// The mapstructure tag with "squash" or explicit omission:
type VerifierProviderConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    APIKey     string  `mapstructure:"api_key" json:"- "
               yaml:"api_key"  // json:"- " excludes from marshaling
    BaseURL    string  `mapstructure:"base_url"`
    // ...
}

// internal/verifier/client.go
// Keys are sent as HTTP headers only:
func (c *Client) authHeader(req *http.Request) {
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
}
```

10.3 Faulty Key Tracking

Mirror HelixAgent's `api_keys.ReadFaultyAPIKeys()` pattern [33](#):

```
// internal/verifier/faulty_keys.go
var faultyKeysPath = filepath.Join(os.Getenv("HOME"),
    ".helixcode", "faulty_api_keys.json")

type FaultyKeyStore struct {
    Keys map[string]FaultyKeyEntry `json:"keys"`
}

type FaultyKeyEntry struct {
    KeyName    string  `json:"key_name"`
    Reason     string  `json:"reason"`
    Timestamp  time.Time `json:"timestamp"`
    Provider   string  `json:"provider"`
}

func MarkKeyFaulty(keyName, reason, provider string) error {
```

```

    store, _ := ReadFaultyKeys()
    store.Keys[keyName] = FaultyKeyEntry{
        KeyName: keyName, Reason: reason, Timestamp: time.Now(),
        Provider: provider,
    }
    data, _ := json.Marshal(store)
    return os.WriteFile(faultyKeysPath, data, 0600)
}

func ReadFaultyKeys() (*FaultyKeyStore, error) {
    data, err := os.ReadFile(faultyKeysPath)
    if err != nil {
        return &FaultyKeyStore{Keys:
            map[string]FaultyKeyEntry{}}, nil
    }
    var store FaultyKeyStore
    json.Unmarshal(data, &store)
    if store.Keys == nil { store.Keys =
        map[string]FaultyKeyEntry{} }
    return &store, nil
}

```

11. HelixAgent Integration

11.1 Integration Model

HelixAgent is a **Git submodule** of HelixCode (confirmed by `cli_agents/helix_code/` being a submodule in HelixAgent's repo [33](#)). The integration is **bidirectional config sharing**, not direct function calls.

11.2 HelixAgent as Submodule

File: `helix_code/.gitmodules` (add if not present)

```

[submodule "HelixAgent"]
    path = HelixAgent
    url = https://github.com/HelixDevelopment/HelixAgent.git

```

11.3 Config Synchronization

HelixCode writes a shared config file that HelixAgent reads:

```

// internal/helixagent/sync.go
package helixagent

import (
    "dev.helix.code/internal/config"
    "gopkg.in/yaml.v3"

```

```

    "os"
    "path/filepath"
)

// SyncConfigToHelixAgent writes a verifier config that
// HelixAgent can consume.
func SyncConfigToHelixAgent(cfg *config.Config) error {
    if cfg.HelixAgent == nil || !cfg.HelixAgent.Enabled {
        return nil
    }

    // Translate HelixCode config to HelixAgent verifier.yaml
    // format
    agentCfg := AgentVerifierConfig{
        Enabled: cfg.Verifier != nil && cfg.Verifier.Enabled,
        Database: AgentDatabaseConfig{
            Path: filepath.Join(cfg.HelixAgent.Path, "data",
                "llm-verifier.db"),
        },
        Providers: make(map[string]AgentProviderConfig),
    }

    for name, p := range cfg.Verifier.Providers {
        agentCfg.Providers[name] = AgentProviderConfig{
            Enabled: p.Enabled,
            APIKey:    p.APIKey,
            BaseURL:  p.BaseURL,
            Models:   p.Models,
        }
    }

    data, err := yaml.Marshal(agentCfg)
    if err != nil {
        return err
    }

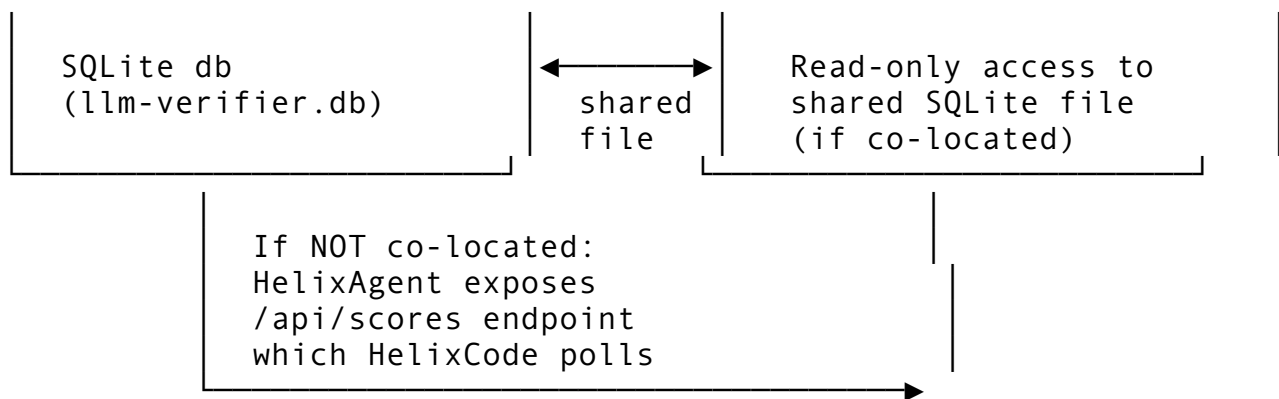
    configPath := filepath.Join(cfg.HelixAgent.Path, "configs",
        "verifier.yaml")
    return os.WriteFile(configPath, data, 0644)
}

```

11.4 Score Sharing

When both systems run with `verifier_sync.share_scores: true`:





11.5 Enable/Disable HelixAgent from HelixCode Config

Behavior Matrix:

helix_agent.enabled	verifier.enabled	Behavior
false	any	HelixAgent submodule is NOT initialized. HelixCode uses its own verifier client directly.
true	true	HelixAgent is initialized. HelixCode's verifier config is synced to HelixAgent. HelixCode reads scores FROM HelixAgent's verifier (via shared DB or REST API).
true	false	HelixAgent verifier is configured but disabled in HelixAgent's own config. HelixCode does not use LLMsVerifier.

11.6 Auto-Start

```
// cmd/server/main.go or cmd/cli/main.go
func maybeStartHelixAgent(cfg *config.Config) error {
```

```

if cfg.HelixAgent == nil || !cfg.HelixAgent.Enabled || !
    cfg.HelixAgent.AutoStart {
    return nil
}

// 1. Sync config
if err := helixagent.SyncConfigToHelixAgent(cfg); err != nil
{
    return fmt.Errorf("failed to sync config to HelixAgent:
    %w", err)
}

// 2. Start HelixAgent as subprocess
cmd := exec.Command(
    filepath.Join(cfg.HelixAgent.Path, "cmd", "helixagent",
    "main.go"),
    "--verifier", "--config",
    filepath.Join(cfg.HelixAgent.Path, "configs",
    "verifier.yaml"),
)
cmd.Env = os.Environ()
if err := cmd.Start(); err != nil {
    return fmt.Errorf("failed to start HelixAgent: %w", err)
}

// 3. Store process for graceful shutdown
globalHelixAgentProcess = cmd.Process

return nil
}

```

12. Score Adapter (The Critical Bridge)

12.1 File: internal/verifier/adapter.go

This is the **most important file** — it bridges LLMsVerifier's scoring system to HelixCode's provider and model selection logic. It replicates HelixAgent's internal/services/llmsverifier_score_adapter.go [33](#).

```
package verifier
```

```
import (
    "context"
    "sync"
    "time"
)
```

```

// Adapter bridges LLMsVerifier scores to HelixCode's llm
package.
type Adapter struct {
    client      *Client
    cache       *Cache
    health      *HealthMonitor
    config       *config.VerifierConfig

    providerScores map[string]float64
    modelScores    map[string]float64
    modelCodeScores map[string]float64
    modelRelScores map[string]float64
    mu             sync.RWMutex

    lastRefresh      time.Time
    refreshInterval time.Duration
}

func NewAdapter(client *Client, cache *Cache, health
    *HealthMonitor, cfg *config.VerifierConfig) *Adapter {
    return &Adapter{
        client:      client,
        cache:       cache,
        health:      health,
        config:       cfg,
        providerScores: make(map[string]float64),
        modelScores:    make(map[string]float64),
        modelCodeScores: make(map[string]float64),
        modelRelScores: make(map[string]float64),
        refreshInterval: cfg.CacheTTL,
    }
}

func (a *Adapter) IsEnabled() bool {
    return a.config != nil && a.config.Enabled
}

func (a *Adapter) IsReachable() bool {
    return a.health.AllowRequest()
}

// GetModelScore returns the overall verifier score (0-10) for a
// model.
// Mirrors HelixAgent's
// llmsverifier_score_adapter.go:GetModelScore() [^33^].
func (a *Adapter) GetModelScore(modelID string) (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()

```



```

    score, ok := a.modelScores[modelID]
    if ok {
        if score > 10 {
            score = score / 10.0
        }
        return score, true
    }

    // Try cache
    if cached, found := a.cache.GetModelScore(modelID); found {
        return cached, true
    }

    return 0, false
}

// GetProviderScore returns the best score for any model of this
// provider.
// Mirrors HelixAgent's GetProviderScore() [^33^].
func (a *Adapter) GetProviderScore(providerType string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()

    score, ok := a.providerScores[providerType]
    if ok {
        if score > 10 {
            score = score / 10.0
        }
        return score, true
    }
    return 0, false
}

func (a *Adapter) GetModelCodeCapabilityScore(modelID string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()
    score, ok := a.modelCodeScores[modelID]
    return score, ok
}

func (a *Adapter) GetModelReliabilityScore(modelID string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()
    score, ok := a.modelRelScores[modelID]

```

```

        return score, ok
    }

func (a *Adapter) GetMinAcceptableScore() float64 {
    if a.config == nil {
        return 6.0
    }
    return a.config.Scoring.MinAcceptableScore
}

// GetVerifiedModels returns all models from verifier, filtered
// by config.
func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*VerifiedModel, error) {
    if !a.IsEnabled() {
        return nil, ErrVerifierDisabled
    }

    // Check cache
    if models, ok := a.cache.GetModels("all"); ok {
        return a.filterByProviderConfig(models), nil
    }

    // Fetch from verifier
    models, err := a.client.GetModels(ctx)
    if err != nil {
        return a.handleFetchError(err)
    }

    // Update internal score maps
    a.refreshScores(models)

    // Update cache
    a.cache.SetModels("all", models)

    return a.filterByProviderConfig(models), nil
}

func (a *Adapter) refreshScores(models []*VerifiedModel) {
    a.mu.Lock()
    defer a.mu.Unlock()

    providerBest := make(map[string]float64)

    for _, m := range models {
        a.modelScores[m.ID] = m.OverallScore
        a.modelCodeScores[m.ID] = m.CodeCapabilityScore
        a.modelRelScores[m.ID] = m.ReliabilityScore
    }
}

```

```

        // Track best score per provider
        if current, ok := providerBest[m.Provider]; !ok ||
        m.OverallScore > current {
            providerBest[m.Provider] = m.OverallScore
        }
    }

    a.providerScores = providerBest
    a.lastRefresh = time.Now()
}

func (a *Adapter) filterByProviderConfig(models
    []*VerifiedModel) []*VerifiedModel {
    if len(a.config.Providers) == 0 {
        return models // no overrides – return all
    }

    filtered := make([]*VerifiedModel, 0, len(models))
    for _, m := range models {
        if pc, ok := a.config.Providers[m.Provider]; ok && !
            pc.Enabled {
            continue
        }
        filtered = append(filtered, m)
    }
    return filtered
}

func (a *Adapter) ForceRefresh(ctx context.Context) error {
    a.cache.Invalidate("all")
    _, err := a.GetVerifiedModels(ctx)
    return err
}

```

13. Implementation Phases

Phase 1: Foundation (Week 1)

Task	File(s)	Lines
Add config structs	internal/config/config.go	+120
Add default values	internal/config/config.go	+30
Add env var bindings	internal/config/config.go	+20
Add validation	internal/config/config.go	+25
Create example YAML	config/verifier.yaml	257

Task	File(s)	Lines
Create internal/verifier/types.go	New	150

Phase 2: Client & Cache (Week 1-2)

Task	File(s)	Lines
REST API client	internal/verifier/client.go	385
Two-tier cache	internal/verifier/cache.go	180
Fallback models	internal/verifier/fallback_models.go	50
Faulty key store	internal/verifier/faulty_keys.go	80

Phase 3: Health & Polling (Week 2)

Task	File(s)	Lines
Circuit breaker	internal/verifier/health.go	486
Background poller	internal/verifier/polling.go	220
Event publisher	internal/verifier/events.go	200

Phase 4: Score Adapter (Week 2-3)

Task	File(s)	Lines
Score adapter	internal/verifier/adapter.go	528
Model source bridge	internal/llm/verifier_integration.go	300

Phase 5: Integration Points (Week 3)

Task	File(s)	Lines
Replace CLI model list	cmd/cli/main.go:101-128	~40
Replace fetchExternalModels	internal/llm/ model_discovery.go	~30
Augment SelectOptimalModel	internal/llm/ model_manager.go	~60
Add provider validation	internal/llm/factory.go	~20

Phase 6: HelixAgent Integration (Week 3-4)

Task	File(s)	Lines
Config sync	internal/helixagent/sync.go	120
Auto-start logic	cmd/server/main.go	+40
Submodule setup	.gitmodules	+4

Phase 7: Testing & Challenges (Week 4)

Task	File(s)	Lines
Unit tests for client	internal/verifier/client_test.go	200
Unit tests for cache	internal/verifier/cache_test.go	150
Unit tests for adapter	internal/verifier/adapter_test.go	250
Challenge script	challenges/scripts/llmsverifier_integration_challenge.sh	200

14. Test Strategy

14.1 Test Files to Create

File	Type	Coverage
internal/verifier/client_test.go	Unit	HTTP client, mock server, error paths
internal/verifier/cache_test.go	Unit	LRU eviction, TTL expiry, Redis fallback
internal/verifier/health_test.go	Unit	Circuit breaker state transitions
internal/verifier/polling_test.go	Unit	Tick behavior, change detection, graceful stop
internal/verifier/adapter_test.go	Unit	Score normalization, filtering, fallback chain
internal/verifier/faulty_keys_test.go	Unit	Read/write JSON persistence
internal/llm/verifier_integration_test.go	Unit	Model conversion, capability mapping
tests/e2e/verifier_integration_test.go	E2E	Full flow: config → poll → CLI list → model selection

14.2 Mock Server for Testing

```
// internal/verifier/client_test.go
func TestClientGetModels(t *testing.T) {
    server := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        assert.Equal(t, "/api/models", r.URL.Path)
        assert.Equal(t, "Bearer test-key",
            r.Header.Get("Authorization"))
        json.NewEncoder(w).Encode([]*VerifiedModel{
```

```

        {ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
OverallScore: 9.2},
    ))
}))
defer server.Close()

client := NewClient(server.URL, "test-key", 5*time.Second)
models, err := client.GetModels(context.Background())
assert.NoError(t, err)
assert.Len(t, models, 1)
assert.Equal(t, 9.2, models[0].OverallScore)
}

```

14.3 Challenge Scripts

New File: challenges/scripts/llmsverifier_integration_challenge.sh

```

#!/bin/bash
# CONST-035 Anti-Bluff: Verify that model listing is NOT
# hardcoded
# This validates BLUFF-002 is permanently fixed

set -e

# 1. Build HelixCode with verifier enabled
make build

# 2. Start LLMsVerifier mock server
python3 tests/mock_verifier_server.py &
VERIFIER_PID=$!
trap "kill $VERIFIER_PID" EXIT

# 3. Run helix --list-models
OUTPUT=$(./bin/cli --list-models)

# 4. Assert output contains dynamic indicators (SC: score suffix)
if ! echo "$OUTPUT" | grep -q "SC:"; then
    echo "FAIL: Model list missing verifier score suffix (SC:).
        BLUFF-002 may be regressed."
    exit 1
fi

# 5. Assert no hardcoded-only models
if echo "$OUTPUT" | grep -q "llama-3-8b.*llama.cpp.*8192"; then
    echo "FAIL: Hardcoded model format detected."
    exit 1
fi

echo "PASS: Model list is dynamically sourced from LLMsVerifier."

```

15. Summary of Exact Modifications

15.1 Existing Files to Modify

File	Line Range	Change
helix_code/internal/config/config.go	After line 253	Add Verifier *VerifierConfig and HelixAgent *HelixAgentConfig fields
helix_code/internal/config/config.go	In setDefaults()	Add all verifier defaults (12+ SetDefault calls)
helix_code/internal/config/config.go	After env bindings	Add BindEnv for verifier API key and all provider API keys
helix_code/internal/config/config.go	In validateConfig()	Add verifier config validation (mode, endpoint, weights sum)
helix_code/cmd/cli/main.go	Lines 101-128	Replace handleListModels() hardcoded list with verifier-aware dynamic fetch
helix_code/cmd/cli/main.go	Flag declarations	Add --refresh-models flag
helix_code/cmd/cli/main.go	NewCLI() function	Initialize verifier adapter if config enabled
helix_code/cmd/server/main.go	After server.New()	Initialize verifier adapter and poller; start HelixAgent if configured
helix_code/internal/llm/model_discovery.go	fetchExternalModels()	Replace hardcoded return with verifier adapter call
helix_code/internal/llm/model_discovery.go	Struct definition	Add verifierAdapter *verifier.Adapter field
helix_code/internal/llm/model_manager.go	SelectOptimalModel()	Insert rankByVerifierScores() call after task type filter
helix_code/internal/llm/model_manager.go	After struct definition	Add verifierAdapter field and rankByVerifierScores() method
helix_code/internal/llm/factory.go	In NewProvider()	Add verifier status validation before returning provider
helix_code/internal/llm/missing_types.go	ModelInfo struct	Add Verified bool, Score float64, Source string fields
helix_code/.env.example	After existing env vars	Add HELIX_VERIFIER_API_KEY,

File	Line Range	Change
		HELIX_VERIFIER_ENDPOINT, and all provider keys
helix_code/go.mod	After existing requires	Add github.com/hashicorp/golang-lru/v2 (already present per 3 , verify)

15.2 New Files to Create

File	Purpose
helix_code/internal/verifier/client.go	REST API client for LLMsVerifier
helix_code/internal/verifier/cache.go	Two-tier LRU + Redis cache
helix_code/internal/verifier/health.go	Circuit breaker health monitor
helix_code/internal/verifier/polling.go	Background polling goroutine
helix_code/internal/verifier/events.go	Event publisher for HelixCode event bus
helix_code/internal/verifier/adaptor.go	Score adapter bridge (critical)
helix_code/internal/verifier/types.go	Shared data types
helix_code/internal/verifier/fallback_models.go	CONST-035 compliant fallback list
helix_code/internal/verifier/faulty_keys.go	Faulty API key tracking
helix_code/internal/verifier/doc.go	Package documentation
helix_code/internal/llm/verifier_integration.go	Bridge between verifier and model discovery
helix_code/internal/helixagent/sync.go	Config sync to HelixAgent submodule
helix_code/config/verifier.yaml	Example configuration file
helix_code/pkg/sdk/verifier/client.go	Public SDK client (copy from HelixAgent pattern)
helix_code/challenges/scripts/llmsverifier_integration_challenge.sh	CONST-035 challenge script
helix_code/internal/verifier/client_test.go	Unit tests
helix_code/internal/verifier/cache_test.go	Unit tests
helix_code/internal/verifier/health_test.go	Unit tests

File	Purpose
helix_code/internal/verifier/ adapter_test.go	Unit tests
helix_code/internal/llm/ verifier_integration_test.go	Unit tests

16. Risk Assessment

Risk	Likelihood	Impact	Mitigation
LLMsVerifier API is minimal (only 5 endpoints wired) 28	High	High	Implement missing endpoints in LLMsVerifier first, or mock for HelixCode testing
Go version mismatch (HelixCode 1.24.0 vs LLMsVerifier 1.25.3)	Medium	Low	REST API decouples versions; no module import needed
digital.vasic.llmprovider sibling dependency	High	High	Use REST API, NOT Go module import
LLMsVerifier core verification is stub (returns 8.5) 28	High	Medium	Accept stub initially; scores will improve as verifier matures. Document in AGENTS.md.
Redis unavailable in some deployments	Medium	Medium	L1 LRU cache operates independently; fallback to memory-only
Circular dependency with HelixAgent submodule	Low	High	HelixAgent is optional; HelixCode verifier works standalone
CLI still uses flag not Cobra	Medium	Low	Continue using flag; verifier integration doesn't require Cobra

17. Citations

-
- ^[10]: helix_code/cmd/cli/main.go — Hardcoded model list (BLUFF-002) at lines 101-128
-

-
-
-
-
-
-
-
-

End of Integration Architecture Document